

Planificación automática aplicada al Senku

*Inteligencia Artificial – Ingeniería del Software
Curso 2025/2026*

Propuesta de trabajo del profesor Gabriel Chaves Benítez

1. Introducción y objetivos

El tema del trabajo se enmarca en el uso de algoritmos de búsquedas para resolver problemas de planificación. En las distintas entregas se explorarán tanto algoritmos basados en heurísticas como algoritmos estocásticos.

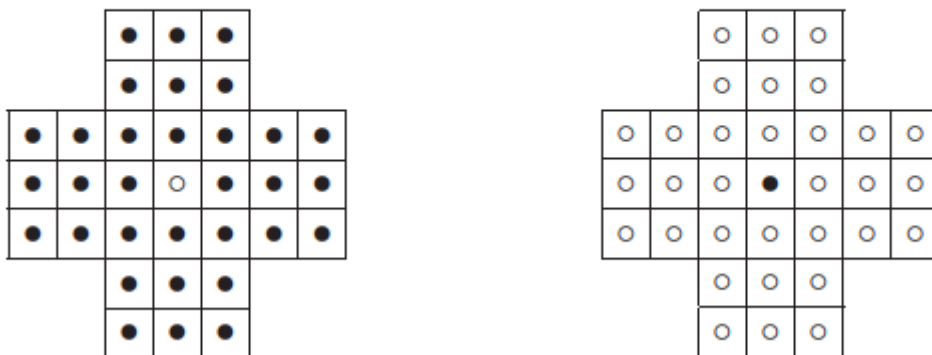
El **objetivo principal** de esta propuesta es ser capaz de diseñar un sistema que consiga representar el juego de senku, también conocido como solitario (mediante PDDL o algún lenguaje similar como GDL) y dar con una secuencia de acciones que lleven a la solución de este. A continuación se intentará ampliar el número de tableros solucionables mediante alguno de los algoritmos que desarrollaremos posteriormente.

Para ello será necesario alcanzar los siguientes **objetivos específicos**:

1. Expresar el problema de manera correcta.
2. Diseñar un sistema capaz de usar este lenguaje y sea capaz de solucionar el tablero los tableros básicos.
3. Aumentar el número de tableros capaces de solucionar aplicando el algoritmo correspondiente según la convocatoria.
4. Documentar el trabajo realizado usando un formato de artículo científico.
5. Realizar una presentación (PDF, PowerPoint o similar).

2. Senku

El juego Senku se enmarca dentro de los juegos de un jugador (o puzzles), este tipo de juegos son de gran utilidad en el ámbito de la IA, ya que permite la evaluación del desempeño de agentes y algoritmos.



Posiciones de inicio y final del tablero clásico 2-dimensional

El Senku comienza con un tablero en el que existen M piezas y un hueco disponible. Cada turno el jugador escoge una de las piezas y, saltando otra piezas vertical u horizontal, pasa su posición a la del hueco; eliminado la pieza que ha sido saltada.

El objetivo final del Senku es terminar con una única pieza en el tablero en la posición del hueco original. Aunque existen versiones más relajadas del problema.

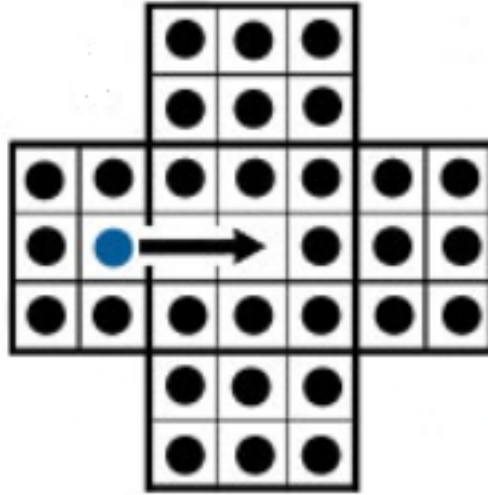


Figura 1: Ejemplo de movimiento válido

Una partida de Senku se puede representar como un árbol en el que los nodos son los posibles estados del juego y las aristas las acciones válidas que nos llevan de un estado a otro. Debido a esto los algoritmos basados en grafos pueden ser una buena herramienta para la exploración del problema.

2.1. Función Pagoda

La heurística que se empleará para este trabajo es la llamada función pagoda. Para evaluar la función de pagoda previamente se le asigna una puntuación a cada casilla del tablero, cumpliendo que para cualesquiera casillas adyacentes a , b , c se da que $a + b \geq c$. Así, para un estado dado, el valor de pagoda será la suma de cada una de las casillas ocupada.

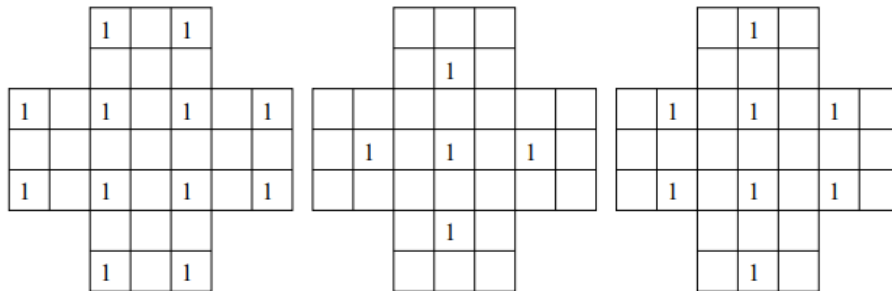


Figura 2: Tres posibles funciones de pagoda para el tablero clásico 2-dimensional

3. Descripción del trabajo

A continuación se describe con más detalle cómo debe llevarse a cabo el trabajo. Este tendrá una parte común para todas las convocatorias y una parte específica para la convocatoria de junio y julio, en la convocatoria de octubre el alumno es libre de escoger una de las partes específicas anteriores.

3.1. Metodología

Para aprobar el alumnado debe entregar tanto la parte común como la parte correspondiente a la convocatoria en la que se presente.

3.1.1. Parte común

En primer lugar debe expresarse el juego como un problema de planificación mediante algún lenguaje específico para ello. A continuación se implementará en Python un sistema capaz de explorar el problema, es decir, partiendo del problema anterior haga una exploración del básica del árbol de juego y sea capaz de encontrar y devolver una consecución de acciones y estados que den lugar al estado objetivo. En esta primera búsqueda será necesario ser capaz de resolver, al menos, el tablero clásico 2-dimensional.

3.1.2. Convocatoria de junio

Además, los alumnos que se presenten a esta convocatoria deberán crear un sistema capaz de recibir dos archivos *.pddl*, uno con el dominio y otro con una instancia del problema y, a partir de estos, ser capaz de navegar entre acciones y estados, pudiendo así, encontrar una solución mediante una búsqueda en espacio de estados utilizando el algoritmo *beam search*:

1. El algoritmo comienza prefijando un valor β que será la “anchura” del rayo.
2. Dado un estado se calculan todos los estados accesibles desde este y se toman los β mejores, de acuerdo con la heurística.
3. Si alguno de los estados accesibles es el estado objetivo, se termina y se devuelve la secuencia de acciones.
4. En otro caso se calculan todos los estados sucesores a los β estados tomados anteriormente y se vuelve a tomar otros β estados distintos.

Con este algoritmo se consigue efectuar una búsqueda en anchura pero descartando los estados “menos prometedores”.

3.1.3. Convocatoria de julio

Los alumnos que se presenten a esta convocatoria deben crear un sistema capaz de recibir dos archivos *.pddl*, uno con el dominio y otro con una instancia del problema y, a partir de estos, ser capaz de navegar entre acciones y estados, pudiendo así, encontrar una solución mediante una búsqueda en espacio de estados utilizando el algoritmo IDA*:

1. El algoritmo en todo el proceso dependerá de la función

$$f(n) = g(n) + h(n)$$

siendo n un estado cualquiera, g la función de coste de haber llegado al estado n y h la función que devuelve el valor de la heurística del estado n .

2. El algoritmo comienza estableciendo el límite m de coste de un estado como el valor de f del estado inicial.
3. Posteriormente calcula todos los posibles estados sucesores y elimina aquellos estados con un valor de $f(n)$ mayor a m .
4. Se elige como estado sucesor aquel que tenga un $f(n)$ menor manteniendo el valor de m . Este proceso continua mientras existan nodos sucesores tales que $f(n) < m$ hasta encontrar solución.
5. En caso de no haber estados sucesores que cumplan que $f(n) < m$ se reinicia la búsqueda desde el principio, tomando como m el menor valor de f de los sucesores del estado inicial.

Con este algoritmo se realiza una búsqueda en profundidad basada en el algoritmo A*, pero con mejor uso de la memoria.

3.2. Variantes del juego.

Indistintamente a la convocatoria que se presente el alumno su ampliación debe ser capaz de resolver la primera, la tercera y la quinta de las variantes presentadas más abajo.

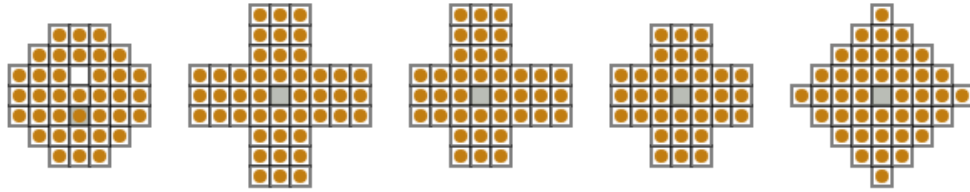


Figura 3: Distintas variantes del Senku

4. Lenguaje de programación y bibliotecas recomendadas

Para la realización del trabajo debe utilizarse el lenguaje de programación Python y se podrá hacer uso de todas las librerías vistas en la asignatura. Y para la especificación del dominio se podrá emplear el lenguaje deseado por el alumno, aunque en caso de no ser PDDL se deberá explicar, brevemente, la sintaxis de este en la documentación del trabajo. Además, para este trabajo específicamente se recomienda:

- Unified Planning es una librería similar a Py2PDDL, pero con mayor flexibilidad en su sintaxis y mayor variedad de solver.
- Fast Downward permite resolver problemas de dominio planteado en PDDL. Recomendado para automatizar la comprobación de resultados.
- ENHSP Similar al anterior pero orientado a la solución de problemas numéricos.

5. Documentación y entrega

El trabajo deberá documentarse siguiendo un formato de artículo científico, con una **extensión mínima de 6 páginas**. En la página web de la asignatura se pueden encontrar plantillas donde se sugiere una estructura general. Estas plantillas siguen el formato de los IEEE conference proceedings, en cuyo sitio web la guía para autores ofrece información más detallada. El documento entregado deberá estar en formato PDF. Se valorará el uso del sistema $\text{\LaTeX}$.

En el caso concreto de este trabajo, la memoria deberá al menos incluir: introducción; descripción del lenguaje de descripción de dominio elegido (si procede); descripción de los algoritmos implementados, explicando las dificultades encontradas y las decisiones de diseño adoptadas para abordarlas; experimentación y descripción de los resultados alcanzados; conclusiones y bibliografía. En ningún caso debe incluirse código en la memoria.

La entrega del trabajo consistirá de un **único fichero comprimido zip** conteniendo la memoria del trabajo, los archivos *.pddl* de dominio y de los distintos problemas y el código implementado (ficheros *py* o cuadernos Jupyter).

6. Presentación y defensa

Como parte de la evaluación del trabajo se deberá realizar una defensa del mismo, para lo que se citará a los alumnos de manera conveniente.

Al inicio de la defensa se deberá realizar una pequeña presentación (PDF, PowerPoint o similar) de diez minutos en la que participarán activamente todos los miembros del grupo que han desarrollado el trabajo. Esta presentación deberá seguir a grandes rasgos la misma estructura que la memoria del trabajo, haciendo especial mención a los resultados obtenidos y al análisis de los mismos.

En los siguientes diez minutos de la defensa el profesor procederá a realizar preguntas sobre el trabajo, que podrán ser tanto de la memoria como del código fuente.

7. Evaluación del trabajo

Para la evaluación del trabajo se tendrán en cuenta los siguientes criterios, considerando una nota total máxima de 4 puntos:

- *Memoria del trabajo* (hasta 2 puntos): se valorará la claridad de las explicaciones, el razonamiento de las decisiones, el análisis y presentación de resultados y el correcto uso del lenguaje. La elaboración de la memoria debe ser original, por lo que no se evaluará el trabajo si se detecta cualquier copia del contenido.
- *Lenguaje y expresión del problema* (hasta 0.5 puntos): se valorará la adecuada parametrización del problema.

- *Búsqueda de la solución* (hasta 0.75 puntos): se valorará la implementación realizada tanto la básica como la extensión del trabajo en función de la convocatoria, también se valorará positivamente, aunque no es necesaria, la mejora y ampliación de los algoritmos.
- *Código fuente* (hasta 0.75 punto): se valorará la claridad y buen estilo de programación, corrección y eficiencia de la implementación y calidad de los comentarios. El código debe ser original, por lo que no se evaluará el trabajo si se detecta código copiado o descargado de internet.
- *Presentación y defensa*: se valorará la claridad de la presentación y la buena explicación de los contenidos del trabajo así como, especialmente, las respuestas a las preguntas realizadas por el profesor, lo que dará lugar, **para cada alumno por separado**, a un factor multiplicativo en el intervalo $[0, 1]$ de la nota total obtenida a partir de los apartados anteriores.

IMPORTANTE: cualquier plagio, compartición de código o uso de material que no sea original y del que no se cite convenientemente la fuente, significará automáticamente la calificación de cero en la asignatura para todos los alumnos involucrados. Por tanto, a estos alumnos no se les conserva, ni para la actual ni para futuras convocatorias, ninguna nota que hubiesen obtenido hasta el momento. Todo ello sin perjuicio de las correspondientes medidas disciplinarias que se pudieran tomar.

Uso de inteligencia artificial generativa

El uso de sistemas de inteligencia artificial generativa está permitido con las siguientes condiciones:

- Debe explicarse para qué se han utilizado esos sistemas, así como describir las entradas (*prompts*) proporcionadas a los mismos.
- En la defensa del trabajo **ambos alumnos** deben demostrar conocimiento y entendimiento de la **totalidad del trabajo**, lo que incluye las respuestas obtenidas de esos sistemas.